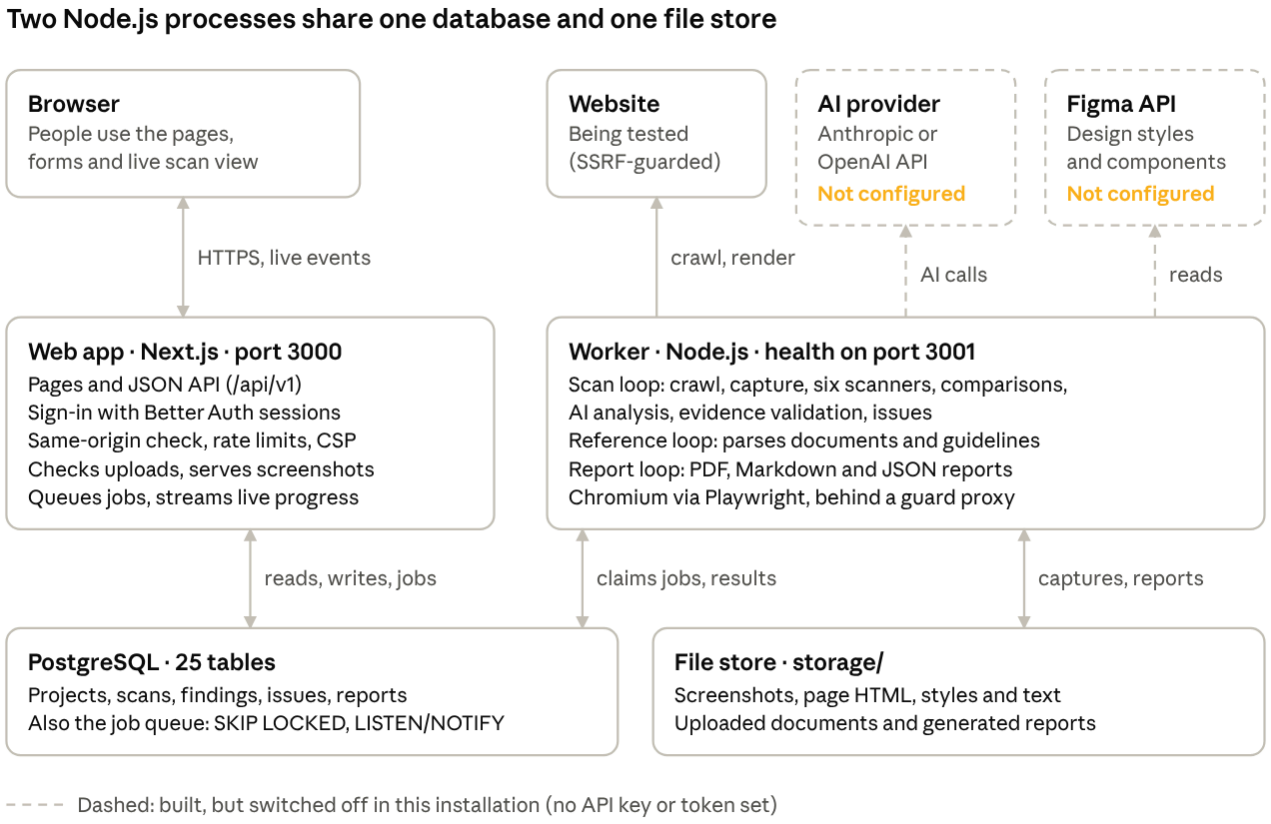
QA Copilot Architecture

Oct 6, 2026 · @INTARCS

QA Copilot is a self-hosted web app that scans a website with a real browser, runs six evidence-based QA checks plus design and content comparisons, and turns the findings into tracked issues and reports. It runs as two Node.js processes, the Next.js web app and a background worker, which share one PostgreSQL database and a local file store.

Two integrations are built but switched off in this installation: the AI layer (no AI provider key is set) and the Figma API (no Figma token is set). Without them every scan still works; issues use the scanners' own wording and design values are entered by hand.

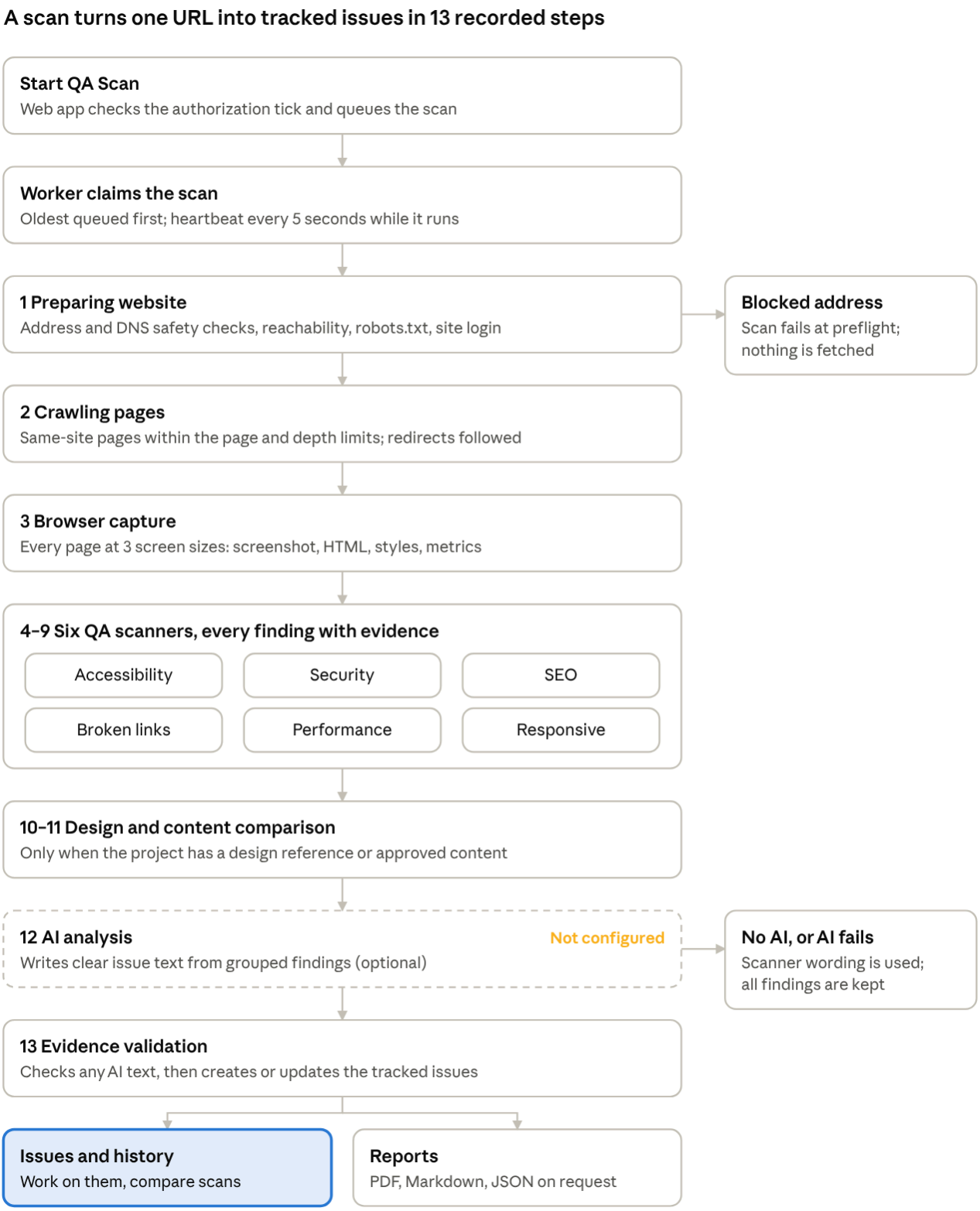
At a glance



System architecture · 2 processes, 1 database, 1 file store, 3 outside services

People only talk to the web app. The web app and the worker never call each other: they meet in the database, where the web app queues jobs and the worker claims them, and in the file store. Only the worker reaches outside, to the website being tested and, when configured, the AI provider and Figma.

How a scan flows



Scan pipeline · 13 steps from Start QA Scan to issues and reports

Every step records its status, reason and tool. A step that fails is reported, and the later steps still run on what was stored, so one problem never hides the rest of the results.

Technology stack

Everything is TypeScript on Node.js, so the UI, the API, the worker and the shared logic use one language and share types. Versions are the ones pinned in the repository's `package.json` files.

Layer	What we used (version)	Why we chose it
Language and runtime	TypeScript 6.0 on Node.js 22.12+	One typed language end to end; types are shared between the web app, worker and packages
Code organisation	npm workspaces (monorepo)	Two apps and ten packages live in one repository and import each other without publishing
Web framework	Next.js 16.3 (App Router) with React 19.3	Pages and the JSON API run in one process; server rendering keeps data on the server; `proxy.ts` adds per-request security headers
Styling and icons	Tailwind CSS 4.3, lucide-react	Utility classes keep the UI consistent without a heavy component library
Sign-in	Better Auth 1.7 (email and password)	A maintained library instead of hand-written password and session code; sessions stored in PostgreSQL; built-in rate limits
Validation	Zod 4.6	One schema style checks environment settings, API request bodies and AI answers
Database	PostgreSQL 15 or newer (developed on 18)	Relational data with real constraints (for example one active scan per project); also serves as the job queue, so no extra queue server is needed
Database access	Drizzle ORM 0.45, drizzle-kit 0.31	Typed queries close to SQL; migrations are plain SQL files that can be reviewed

Layer	What we used (version)	Why we chose it
Running TypeScript	tsx 4.23	Runs the worker and scripts directly from TypeScript, no build step
Browser automation	Playwright 1.63 with Chromium	A real browser renders each page, so screenshots, computed styles, layout and performance are measured as a visitor sees them
Crawling	undici 7.30 (HTTP), cheerio 1.2 (HTML), robots-parser 3.0	Fast HTTP fetching with full control over redirects and connections; HTML parsing without a browser; correct robots.txt rules
Accessibility engine	axe-core 4.13 via @axe-core/playwright	The industry-standard WCAG rule engine
Design maths	culori 4.0	Colour difference measured as CIEDE2000, the standard for how different two colours look
Content diff	diff 9.0	Word-by-word differences between approved and live text
Sitemap parsing	fast-xml-parser 5.11	Reads and validates sitemap.xml for the SEO scanner
Document parsing	unpdf 1.8 (PDF), mammoth 1.13 (DOCX), mdast-util-from-markdown 2.1 (Markdown)	Turn uploaded approved content and brand guidelines into structured text
AI providers (optional)	@anthropic-ai/sdk 0.131; an OpenAI-compatible client built on fetch	Claude through the official SDK, or any OpenAI-compatible API; both behind one interface
Network safety	ipaddr.js 2.5	Classifies IP addresses so private, loopback and cloud-metadata addresses can be blocked
Logging	pino 10.4	Structured JSON logs with secrets redacted
Testing and quality	Vitest 5.0, Playwright Test 1.63, ESLint 9.39, Prettier 3.9	Unit, integration and end-to-end tests; consistent code style

Repository layout

The code is split into two runnable apps and ten shared packages. The web app only uses the four packages it needs to serve pages and data; all heavy work (browser, scanners, AI, PDF rendering) lives in packages that only the worker loads.

Folder	What it owns	Used by
`apps/web`	The user interface (pages), the JSON API under `/api/v1`, sign-in, rate limits and security headers	People's browsers
`apps/worker`	Background jobs: scans, document parsing, report rendering; worker health endpoint	Runs on its own
`packages/core`	Settings (env validation), logging, error types, domain constants, scan comparison logic	Web, worker, all packages
`packages/db`	Database schema (25 tables), migrations, repositories (all SQL lives here)	Web, worker
`packages/security`	URL and network safety (SSRF guard), upload checks, encryption of stored secrets	Web, worker, crawler, browser
`packages/storage`	Reading and writing files under the storage folder, with safe file keys	Web, worker
`packages/crawler`	Fetching pages over HTTP, following links and redirects, robots.txt, technology detection	Worker, scanners
`packages/browser`	Launching Chromium, capturing each page at each screen size, rendering PDF reports	Worker
`packages/scanners`	The six QA scanners plus the design and content comparisons	Worker
`packages/reference`	Reading design references (Figma, manual values, brand guidelines) and approved content documents	Worker

Folder	What it owns	Used by
`packages/ai`	Grouping findings, prompts, AI providers, evidence validation (grounding), issue text	Worker
`packages/reports`	Building the PDF (HTML), Markdown and JSON reports	Worker
`fixtures/`	The demo website with known defects (port 4100) and sample reference documents	Tests, demos
`tests/`	Integration test setup, end-to-end tests, smoke tests	Developers
`scripts/`	One-off setup: create `.env`, bootstrap the database role	Developers
`docs/`	The Phase 0 architecture plan and the deployment guide	Readers
`storage/`	Screenshots, page snapshots, uploads and reports (not in git)	Web, worker

Web app (`apps/web`)

The web app is the only part people talk to: it serves the pages, answers the API and records what people ask for, but never runs a scan itself. Long work is written to the database as a job and picked up by the worker. It listens on port 3000.

Pages people use

Area	Pages	What they show
Sign-in	`/login`, `/signup`	Email and password sign-in; account creation (can be switched off)
Overview	`/dashboard`, `/projects`, `/scans`, `/issues`, `/reports`, `/settings`	Totals across projects, all lists, account and system status (database, migrations, AI and Figma settings)

Area	Pages	What they show
New project	`/projects/new`	The 6-step wizard: project, Figma / brand, content, configure scan, review, start scan
Inside a project	Overview, Scan, References, Issues, Pages, Scanner results, Reports, Scan history, Settings	One tab each; the scan page shows live progress
Inside a scan	`/scans/[scanId]` with its findings, issues and reference comparison views	Every step's result, every finding with evidence
Comparison	`/history/compare?base=…&target=…`	Two scans side by side: new, resolved, changed and other categories

The JSON API (`/api/v1`)

Every page action goes through about 40 route handlers under `/api/v1`: projects, scans, findings, issues and their statuses, references, content documents, reports and their downloads, artifacts (screenshots), comparisons, history, a dashboard summary and health checks. Each authenticated route is wrapped by one helper (`authedHandler`) that does the same five things in order:

1. Checks that a state-changing request comes from the app's own origin (otherwise 403), which blocks cross-site request forgery.
2. Requires a valid session cookie (otherwise 401).
3. Counts the request against the user's rate limits (otherwise 429 with a Retry-After header).
4. Runs the route, which only ever reads or changes rows owned by that user; anything else is answered 404, so other people's data cannot even be confirmed to exist.
5. Returns errors in one format (`application/problem+json`) with a safe message, and logs the request with a request id.

Sign-in. Better Auth handles accounts under `/api/auth/*`: passwords are hashed, sessions live in PostgreSQL for 7 days and travel in an httpOnly cookie that page scripts cannot read. Signed-out visitors to any app page are redirected to `/login?next=<page>` and sent back after signing in.

Every page response passes through `proxy.ts`, which adds a fresh Content-Security-Policy nonce for each request, so only the app's own scripts can run.

Live scan progress. The scan page opens a Server-Sent Events stream (`/api/v1/scans/:id/events`). The worker announces every change with a PostgreSQL `NOTIFY`; the web app keeps one `LISTEN` connection and forwards each event to the open pages. If the stream drops, the page falls back to polling every 3 seconds.

Uploads (approved content, brand guidelines) are checked in the web app (type, size, real file bytes, DOCX inflation), stored in the file store and queued for the worker to parse.

Worker (`apps/worker`)

The worker does every slow or risky job (browsers, scanners, AI calls, document parsing, PDF rendering) so the web app stays fast and never touches the scanned website. It is a separate Node.js process started with `npm run dev:worker` or `npm run start:worker`.

Three job loops run side by side in one process, so a long scan never delays an upload or a report:

Loop	Picks up	Wake-up signal	Recovers stuck jobs after
Scan loop	Queued scans (one at a time by default, `WORKER_CONCURRENCY`)	`LISTEN scan_queued`	2 minutes without a heartbeat
Reference loop	Uploaded content documents, brand guidelines, Figma references	`LISTEN reference_queued`	5 minutes
Report loop	Requested PDF, Markdown and JSON reports	`LISTEN report_queued`	5 minutes

The queue is the database. A job is a row with a status. A worker claims the oldest queued row in one SQL statement (`UPDATE … FOR UPDATE SKIP LOCKED`), so several workers can run safely without taking the same job. Each loop is woken at once by a PostgreSQL notification and also checks every 5 seconds, in case a notification is missed.

Heartbeats and ownership. While a scan runs, its worker writes a heartbeat every 5 seconds. A scan whose heartbeat stops (a crashed worker, a lost machine) is marked failed by the periodic recovery check every minute, with a reason. Heartbeats and the final result are only accepted from the worker that owns the scan, so a recovered scan cannot be overwritten later by a worker that comes back.

Cancellation. Cancel on a queued scan takes effect at once. For a running scan the web app records the request; the worker sees it on its next heartbeat (within 5 seconds), aborts the current step and marks the remaining steps skipped. Each step has its own abort signal and time limit (`SCAN_SCANNER_TIMEOUT_MS`, 10 minutes by default).

Start-up and shutdown. At start the worker validates its settings, connects to the database and checks migrations; with migrations pending it pauses scan processing and says so. On SIGTERM or SIGINT it stops taking new jobs and gives running scans `WORKER_SHUTDOWN_GRACE_MS` (20 seconds) to finish; after that they are stopped and recorded as failed with the reason "the worker was shut down".

Health endpoint. `http://127.0.0.1:3001/health` reports the database connection and the state of each loop. It listens on localhost only by default (`WORKER_HEALTH_HOST`).

Scan pipeline

A scan is 13 steps run in order by the worker; each step records its own status (pending, running, completed, failed, inconclusive or skipped), a plain-language reason, the tool and version used, and its findings. On the fixture site the whole run takes under a minute (48 seconds for 20 pages at 3 screen sizes).

#	Step	What it does	Tooling	What it stores
1	Preparing website (preflight)	Resolves the address and blocks private or internal targets; checks the site answers; reads robots.txt; detects the technology; prepares the site login if one is set	undici, ipaddr.js	Reachability, technology guess, robots.txt status
2	Crawling pages	Visits same-site pages up to the page and depth limits, follows redirects, removes tracking parameters, obeys robots.txt, and finds links that only JavaScript adds	undici, cheerio, Chromium	One row per page: HTTP status, redirect chain, depth, title, raw HTML

#	Step	What it does	Tooling	What it stores
3	Browser capture	Opens every page at every screen size (desktop 1440×900, tablet 768×1024, mobile 390×844) in Chromium, 3 pages at a time	Playwright	Screenshot, rendered HTML, computed styles, visible text, network requests, console messages, Core Web Vitals, layout measurements
4–9	Six QA scanners	Accessibility, security, SEO, broken links, performance, responsive layout (next section)	See next section	Findings, each with evidence
10	Design comparison	Compares the confirmed design reference with the computed styles of matching elements; runs only when a design reference exists	culori	One finding per difference; every compared value listed
11	Content comparison	Compares approved content blocks with the visible text of the matching page; runs only when parsed documents exist	diff	Matches and mismatches with word diffs
12	AI analysis	Groups findings and asks the AI provider to write developer-ready issue text; skipped with a reason when no provider is set	@anthropic-ai/sdk or OpenAI-compatible API	AI runs, token use, draft issue text

#	Step	What it does	Tooling	What it stores
13	Evidence validation	Checks every AI sentence against the stored evidence, keeps or replaces it, then creates or updates the tracked issues	Built in	Issues, grounding checks, status events

Each step only reads what earlier steps stored, so a failed step does not stop the others: the scan ends **completed**, **completed with failures** (some steps failed), **failed** (it could not run, e.g. a blocked address) or **cancelled**. A check that cannot be decided is recorded as inconclusive or "unable to verify", never as a pass or a problem.

The six QA scanners

Each scanner reads the stored pages and captures, applies its rules and writes findings; every finding carries evidence (an HTML snippet, a header, a measured value, a screenshot) and a verdict: **confirmed**, **potential** (needs review), **inconclusive** or **unable to verify**. All checks are passive: normal page requests only, no attack traffic.

Scanner	What it checks	Tool	Example from the demo site
Accessibility	WCAG 2.2 A and AA rules plus best practices, per page and screen size; visible keyboard focus	axe-core 4.13	Image without alt text on `/about`

Scanner	What it checks	Tool	Example from the demo site
Security	HTTPS and HSTS; security headers (CSP, nosniff, framing, Referrer-Policy, Permissions-Policy); cookie flags; server version disclosure; insecure forms; Subresource Integrity; credentials exposed in page source	Built in	Cookie without HttpOnly; `Server: DemoServer/1.2.3`
SEO	Titles, meta descriptions, headings, indexability, canonical links, `lang`, viewport, image alt, Open Graph, structured data (JSON-LD), duplicates, URL hygiene, robots.txt, sitemap	Built in, fast-xml-parser	Missing meta description; duplicate titles
Broken links / mixed content	Every unique link (HEAD then GET, with retries), redirect chains, failed resources, http content on https pages; separates confirmed, temporary and inconclusive failures	Built in	Links to a 404 page and a 500 page

Scanner	What it checks	Tool	Example from the demo site
Performance	Lab Core Web Vitals (LCP, CLS, approximate TBT), TTFB and FCP against Google's thresholds; page weight, JavaScript and CSS size, large or oversized images, render-blocking resources	Chromium performance APIs	1.5 s server response on `/slow`
Responsive / visual	Horizontal overflow, clipped text, overlapping controls, touch targets, small text, oversized images; a live test of the mobile menu	Built in, measured in Chromium	1100 px banner overflowing on mobile

Projects choose which scanners run (all six by default). The two reference comparisons run automatically when the project has references.

Design and content comparison

References describe what the site should look like and say; the comparisons check the live site against them. Nothing is guessed: a value that cannot be matched to an element on the page is reported as "unable to verify".

Where a design reference comes from (any combination, all optional):

Source	How it is read	Needs
Figma file link	Local text and colour styles used in the file, Button components named Primary and Secondary, a frame named Container	`FIGMA_ACCESS_TOKEN` (not set here)

Source	How it is read	Needs
Brand guideline document (PDF, DOCX, TXT, MD)	Values written literally, such as "Primary — #1A56DB", each kept with a citation to its line; AI-assisted extraction when an AI provider is set	Nothing extra
Manual values	The editor in the wizard or References tab, or a JSON import	Nothing extra

Extracted values are drafts until someone confirms them. Confirming creates a numbered version of the design reference, and each scan compares against the version that was active when it started.

How design is compared. The reference covers typography (headings 1–6, body, links, button text), colours with a role (text, link, header background, page background), primary and secondary buttons, container width and a spacing scale. Each role is mapped to elements on the page: an explicit CSS selector first, then semantic HTML, then class names (a class-name match can only produce a potential mismatch). Colours are compared by CIEDE2000 difference: 2 or less is a match, 2 to 5 needs review, more than 5 is a mismatch. Sizes allow 0.5 px; sizes are compared at the widest screen.

How content is compared. Approved documents are parsed into blocks (heading, paragraph, list item, call to action, link, emphasis) by the worker, in an isolated thread with limits. Each block is matched to a page by, in order: the page set on the upload, a `Page:` or `URL:` marker in the document, the page's name, then heading similarity. Results are exact, changed wording (similarity 0.85 or more, with a word diff), rewritten, missing, capitalisation, and call-to-action text or link target.

Verdicts for both: match, confirmed mismatch, potential mismatch, unable to verify. Every compared value, matches included, is listed on the scan's reference comparison page; only mismatches become findings.

AI layer and evidence validation

Status today: built, not configured. `AI_PROVIDER=none` in this installation, so the AI step is skipped with a reason and every issue uses the scanners' own wording and the rule catalogue's fix guidance. The AI path has only been tested against a local mock of an OpenAI-compatible API, never against the live Anthropic or OpenAI services.

The AI layer is designed as an optional writer, never a source of facts: the scanners find the problems and hold the evidence, and the AI only rewrites them into clearer, developer-ready issue text that is then checked line by line.

How it works, step by step

1. **Grouping (no AI).** Findings of the same rule become one issue group, for example all "meta description missing" findings across pages. Design and content mismatches form one group per reference value. Each group gets a verdict from its findings.
2. **Prompt.** Groups are sent in batches (`AI_CONCURRENCY`, 3 requests at a time by default). The findings travel as JSON inside a marked data section with an explicit instruction to treat it as data, so text scraped from a website cannot steer the model (prompt injection).
3. **Structured answer.** The model must answer in a fixed JSON schema: title, description, severity with a rationale, affected element, expected, actual, recommendation, the source finding ids, and up to 6 claims, each tied to a finding and an evidence field.
4. **Providers.** Anthropic (Claude) through the official SDK, with an optional effort level and an optional server-side refusal fallback; or any OpenAI-compatible API through `OPENAI_BASE_URL`. Server errors are retried; each request has a 5-minute timeout and a 16,000-token output cap.
5. **Evidence validation (grounding).** A deterministic checker decides how much AI text may be shown:
   - Hard checks: the cited finding ids must exist in this scan, belong to this issue and have evidence. A failure **rejects** the AI text and the scanners' wording is used.
   - Soft checks: every number, colour, URL, page path, finding id and quote must appear in the evidence; length limits apply; severity may move at most one level from the scanners' and is never raised for unconfirmed findings; the affected element must be one of the findings' selectors. A failure replaces only that field (**partial**).
   - Affected URLs and the category always come from the scanners, never from the AI.
6. **Outcome per issue:** passed, partial, rejected or not applicable (no AI text). Raw findings and evidence are never changed by the AI.

When the AI fails (outage, timeout, refusal), the step is marked failed with the reason, the scan ends "completed with failures", and every issue is still created from the scanner findings. In testing, a simulated outage left all 98 findings and 53 issues intact.

To switch it on: set `AI_PROVIDER=anthropic`, `ANTHROPIC_API_KEY` and `AI_MODEL` (all three required together) in `.env`, then restart the worker. For OpenAI or a compatible server use `AI_PROVIDER=openai-compatible` with `OPENAI_API_KEY` and `AI_MODEL`; `OPENAI_BASE_URL` defaults to `https://api.openai.com/v1`.

Issues, history and comparison

An issue is one problem tracked across all scans of a project, so its number (ISSUE-12), status and history survive every rescan. Findings are what one scan saw; issues are what people work on.

Two layers make that possible:

- **Issue track** (`issue_track`): one row per project and fingerprint (the scanner rule plus what it applies to). It holds the stable number, the current status, severity, and the first and last scan that saw it.
- **Issue occurrence** (`issue`): one row per scan in which the track was found, with that scan's text, verdict, confidence, AI or scanner origin and grounding result, linked to its findings (`issue_finding`).

Statuses

Status	Set by	Meaning
Open	Scan or person	Confirmed problem, nobody working on it yet
In progress	Person	Someone is fixing it
Resolved	Person	Fixed; scans never resolve an issue on their own
Ignored	Person	Accepted or not relevant
Reopened	System only	It was resolved and a later scan found it again with evidence
Inconclusive	System only	The latest evidence could not decide it

Every change is recorded in `issue_status_event` with who made it, when and an optional note, and appears in the issue's History panel. People can change up to 100 issues at once; the two system statuses cannot be set by hand. Issue lists filter by severity, category, status, verdict, confidence, page URL and text, with sorting and paging.

Scan history and comparison. Any two finished scans of a project can be compared. Each issue lands in one category:

Category	Meaning
New	Found now; the earlier scan checked the same page with the same scanner and did not find it
Reopened	Found again after being resolved, or after a scan that no longer found it

Category	Meaning
Changed	Found in both, but severity, verdict, affected pages or findings differ
Resolved	Gone, and the later scan re-checked every affected page
Persistent	Found in both with the same severity, verdict and findings
Not rechecked	Gone, but the later scan could not check it (scanner off, page failed), so it is not counted as fixed
Newly checked	Only in the later scan, because the earlier one could not have seen it

The comparison also lists scanner and step status changes (for example AI analysis completed in one scan and failed in the next) and warnings when settings differ. Summaries are cached in `scan_comparison`. On the comparison of the latest scan, one click sets the still-open "resolved" issues to Resolved; this is the only way a scan leads to a fix being recorded.

Reports

A report is a downloadable snapshot of one scan, built only from what is stored in the database, in three formats.

Format	Best for	What it holds
PDF	Sharing with developers and stakeholders	Formatted report; up to 25 affected URLs per issue and 20 inconclusive checks per step
Markdown	Tickets, pull requests, wikis	The same content as plain text
JSON (version 1)	Other tools and automation	Everything, including every finding and its full evidence

Every report contains the scan's settings, every scanner and step result (failed, inconclusive and skipped ones too), the checks that could not be verified, and every issue with its status, verdict, fix guidance and source findings.

How one is made: the Reports tab creates a report row with status pending; the worker's report loop picks it up, builds the content (`packages/reports`), and for PDF renders the HTML in a short-lived Chromium (a few seconds). The file is saved under `storage/reports/<scan id>/` and downloaded as an attachment named like `qa-report-<project>-scan-1.pdf` . Reports are made one at a time and limited to 30 requests per user per 10 minutes.

Issue statuses in a report are those at the moment it was generated; generate a new one after changing statuses. A report covers one scan and does not include a comparison.

Data model

All state lives in 25 PostgreSQL tables, defined with Drizzle in `packages/db/src/schema` and created by 7 SQL migrations (`0000` to `0006`). Large files are not stored in the database, only their storage keys.

The chain to remember: **user → project → scan → page → capture → finding → evidence**, and findings → **issues** that people work on.

Area	Table	What one row holds
Accounts (Better Auth)	`user`	A person's account
	`session`	A signed-in browser session (7 days)
	`account`	The sign-in method, including the hashed password
	`verification`	Short-lived verification tokens
Projects	`project`	A website to test: owner, URL, scan defaults (scanners, screen sizes, page and depth limits)
	`project_secret`	Encrypted site login credentials or a Figma token for one project
	`reference_source`	A design reference source (Figma, manual or guideline document) and its processing status
	`design_spec`	A numbered, confirmed version of the design reference
	`content_document`	An uploaded approved-content file and its parsing status

Area	Table	What one row holds
	`content_block`	One parsed block of that document (heading, paragraph, call to action…)
Scans	`scan`	One run: status, settings snapshot, owning worker, heartbeat, cancel request
	`scanner_run`	One step of a scan: status, reason, tool and version, finding count
	`page`	A crawled page: HTTP status, redirect chain, depth, title, raw HTML key
	`page_capture`	One page at one screen size: screenshot, DOM, styles and text keys, metrics, console messages
	`network_request`	A request the browser made while capturing
Findings	`raw_finding`	What a scanner found: rule, severity, verdict, page, element, message, fix guidance
	`evidence`	Proof attached to a finding: HTML snippet, header, measurement, screenshot reference
Issues and AI	`issue_track`	A problem tracked across scans: number, status, severity, first and last seen
	`issue`	That problem as written for one scan: text, verdict, confidence, origin, grounding result
	`issue_finding`	Links an issue to its findings
	`issue_status_event`	One status change: who, when, from, to, note
	`ai_run`	One AI request: provider, model, tokens, outcome
	`grounding_check`	One evidence check on AI text and its result
Output	`report`	A requested report: format, status, storage key, size

Area	Table	What one row holds
	`scan_comparison`	A cached comparison summary of two scans

Rules the database enforces: every project belongs to one user, and every query is scoped to the signed-in user; a unique index allows only one queued or running scan per project; deleting a user removes all their rows by cascade, while a deleted project is soft-deleted (hidden, its rows kept); status fields use fixed PostgreSQL enums (for example severity: critical, high, medium, low, info).

File storage

Large files live on disk under `STORAGE_DIR` (`./storage` by default, not in git); the database keeps only each file's key. The web app and the worker must see the same folder.

Folder	What is inside	Written by	Read by
`storage/scans/<scan id>/`	Raw HTML per page; per page and screen size a PNG screenshot, the rendered HTML, computed styles (JSON) and visible text (JSON)	Worker (crawl and capture)	Scanners; the web app shows screenshots
`storage/projects/<project id>/`	Uploaded approved-content documents and brand guidelines	Web app (upload)	Worker (parsing)
`storage/reports/<scan id>/`	Generated PDF, Markdown and JSON reports	Worker (report loop)	Web app (download)

How files are served safely: screenshots and snapshots go through `/api/v1/artifacts/<key>`, which checks that the signed-in user owns the scan and rejects keys that try to leave the folder (`..`, absolute paths). Stored HTML is always served as plain text with `nosniff`, so a scanned site's HTML can never run inside the app. Reports download as attachments.

Clean-up: files are not removed automatically when rows disappear (for example a deleted account). `npm run storage:gc` lists files that no record references (a dry run); `npm run storage:gc -- --apply` removes them, only those older than 24 hours by default.

Security

The app fetches arbitrary websites on a user's behalf and stores their credentials, so protection is layered: each layer below works even if another one fails. All of these were checked during the handover verification without destructive testing.

Layer	Threat	Protection
Scan targets	The scanner being pointed at internal systems (SSRF)	Only `http` and `https`, no user name or password in the URL; private, loopback, link-local and cloud-metadata addresses are blocked when the URL is saved, again after DNS lookup at preflight, on every redirect and connection, and inside Chromium through a local guard proxy. `SCAN_ALLOWED_PRIVATE_HOSTS` lists the only exceptions (`localhost:4100` for the demo site)
Scan behaviour	Testing sites without permission; harming them	Every scan requires the "I am authorized" confirmation; robots.txt respected by default; passive checks only; site credentials are sent only to the target site, never to CDNs or analytics
Accounts	Password guessing, stolen sessions	Hashed passwords, 10 to 128 characters; httpOnly session cookies (Secure on HTTPS); 10 sign-ins and 5 sign-ups per minute per IP; an account locks for 15 minutes after 10 wrong passwords, whatever IP they come from; sign-up can be closed (`AUTH_ALLOW_SIGNUP=false`)

Layer	Threat	Protection
API	Reading or changing another user's data; forged requests; abuse	Same-origin check on changes (403); session required (401); every query scoped to the owner (404 otherwise); request bodies validated with Zod; size limits (413); per-user rate limits (429): 600 requests a minute, 20 scan starts an hour, 30 new projects an hour, 30 uploads, reports or reference requests per 10 minutes
Pages	Script injection, clickjacking	A per-request nonce Content-Security-Policy (only the app's own scripts run); X-Frame-Options DENY and `frame-ancestors 'none'`; nosniff; Referrer-Policy; Permissions-Policy; HSTS on HTTPS; no X-Powered-By header
Uploads	Malware, oversized files, archive bombs	PDF, DOCX, TXT and Markdown only; 20 MB limit; extension, declared type and real file bytes must agree; DOCX archives are checked for size and actually decompressed to catch entries that lie about their size (200 MB total limit); file names are cleaned; parsing runs in an isolated thread
Stored secrets	Leaked credentials	Site logins and Figma tokens are encrypted with AES-256-GCM (`ENCRYPTION_KEY`) and never shown again; logs redact secrets and strip query strings; error messages shown to users are sanitised
AI	Prompt injection from scanned pages	Page text sent as marked data; fixed answer schema; every AI sentence checked against evidence

Known limits: report downloads and artifacts receive the app-wide security header instead of their stricter one (still safe: attachments, `nosniff`, HTML served as text); Chromium runs without its own sandbox (Playwright's default), so production should isolate the worker; rate limits are kept in the web process's memory, so run a single web instance.

Reliability and operations

The system is built to fail visibly and recover on its own: every job has an owner and a heartbeat, every check reports why it failed, and nothing is silently skipped.

Health checks

Endpoint	Answers	Use it for
`GET /api/v1/health`	Web app is up (status, version)	Liveness probe
`GET /api/v1/health/ready`	Settings valid, database reachable (latency, version), migrations applied (7 of 7), storage writable; `"status":"ready"`	Readiness probe; also shown on the Settings page
`GET http://127.0.0.1:3001/health`	Worker database connection and the state of the scan, reference and report loops	Worker monitoring

Safety nets

- **Settings are validated at start-up.** A missing or malformed variable stops the process with a clear message; risky choices (open sign-up, allowed private hosts) are logged as warnings.
- **Migrations are checked.** With migrations pending, the readiness check and Settings page say so and the worker pauses scans until `npm run db:migrate` runs.
- **One active scan per project** is enforced by a unique database index, so two simultaneous starts give one scan and one HTTP 409.
- **Stuck jobs are recovered** (heartbeats, ownership and recovery timers; see the Worker section), and a stopped worker records why its scans ended.
- **Each step is isolated** with its own time limit and abort signal, so one slow scanner cannot hang the scan.

Logging. Both processes write structured JSON lines (pino) with the service, component, request id and scan id; `LOG_LEVEL` sets the detail. Passwords, tokens, cookies and API keys are redacted, and URLs are logged without their query strings.

Maintenance. `npm run storage:gc` finds and removes stored files that no record references. Back up two things together: the PostgreSQL database and the `storage/` folder (`docs/DEPLOYMENT.md` has the procedure).

Configuration

Both processes read one `.env` file at the repository root, validated by `packages/core/src/config.ts`. `npm run setup:env` creates it with generated secrets; `.env.example` documents every variable. Never commit or share `.env`.

Variable	What it controls	Default
`DATABASE_URL`	PostgreSQL connection for the app (required)	none
`TEST_DATABASE_URL`	Database for integration tests; its name must end in `_test`	none
`BETTER_AUTH_SECRET`	Signs sessions, at least 32 characters (required)	generated
`ENCRYPTION_KEY`	64 hex characters; encrypts site logins and Figma tokens	generated
`APP_URL`	The app's public address; also the only origin allowed to make changes	`http://localhost:3000`
`NODE_ENV`, `LOG_LEVEL`	Mode and log detail	`development`, `info`
`DATABASE_POOL_MAX`	Database connections per process	10
`AUTH_ALLOW_SIGNUP`	Whether anyone can create an account	`true`
`RATE_LIMIT_ENABLED`	Per-user API limits	`true`
`TRUSTED_PROXIES`, `CLIENT_IP_HEADER`	Which reverse proxies to trust for the client IP	none, `x-forwarded-for`
`STORAGE_DIR`, `MAX_UPLOAD_MB`	File store location; upload size limit	`./storage`, 20

Variable	What it controls	Default
`SCAN_MAX_PAGES`, `SCAN_MAX_DEPTH`	Default crawl limits for new projects	25, 3
`SCAN_PAGE_TIMEOUT_MS`, `SCAN_SCANNER_TIMEOUT_MS`	Time limit per page; per step	30 s, 10 min
`SCAN_CONCURRENCY`, `LINK_CHECK_RETRIES`	Pages captured in parallel; retries per link	3, 3
`SCAN_USER_AGENT`, `SCAN_RESPECT_ROBOTS`	How the scanner identifies itself; robots.txt	`QACopilotBot/0.1`, `true`
`SCAN_ALLOWED_PRIVATE_HOSTS`	Internal hosts that may be scanned (`host:port` list)	empty; `.env.example` sets `localhost:4100`
`AI_PROVIDER`	`none`, `anthropic` or `openai-compatible`	`none`
`AI_MODEL`	Model name, required with a provider	none
`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `OPENAI_BASE_URL`	Provider credentials and endpoint	none
`AI_CONCURRENCY`, `AI_TIMEOUT_MS`, `AI_MAX_OUTPUT_TOKENS`	Parallel AI requests; timeout; output cap	3, 5 min, 16,000
`AI_EFFORT`, `AI_REFUSAL_FALLBACK`, `AI_TEMPERATURE`	Anthropic effort level; refusal fallback; OpenAI temperature	model default, `true`, 0
`FIGMA_ACCESS_TOKEN`	Server-wide Figma token (projects can also store their own)	none
`WORKER_HEALTH_PORT`, `WORKER_HEALTH_HOST`	Worker health endpoint	3001, `127.0.0.1`
`WORKER_CONCURRENCY`, `WORKER_SHUTDOWN_GRACE_MS`	Scans per worker; time to finish on shutdown	1, 20 s

Variable	What it controls	Default
`DEMO_MODE_ENABLED`, `DEMO_SITE_PORT`	The demo fixture site	`true`, 4100

In this installation the AI and Figma variables are empty, which is why those features are off.

Testing and quality

All automated tests passed in the latest runs: 506 of 506 unit and integration tests (6 Oct 2026), 10 of 10 end-to-end tests and all 38 smoke-test checks (5 Oct 2026).

Level	Command	What it covers	Size and time
Static checks	`npm run typecheck`, `npm run lint`, `npm run format:check`	Types across all packages, lint rules, formatting	Seconds
Unit and integration	`npm test`	Every package's logic; integration tests use a real PostgreSQL test database (cleared between tests) and real Chromium scans of the demo site	53 files, 506 tests, about 5 minutes
End-to-end	`npm run build`, then `npm run test:e2e`	The real UI in a browser: accessibility audits (desktop and 390 px mobile), a full scan, references and comparison, the issue workflow with reports, scan history across v1 and v2, and an acceptance run of the whole journey with the security policy checked on every page	10 tests, about 3 minutes; own ports 3100 and 3191

Level	Command	What it covers	Size and time
Smoke test	`npm run smoke`	A running installation: health, sign-in protection, a project, scans of v1 and v2, findings with evidence, issues, a JSON report, history and comparison; deletes its test account afterwards	38 checks, about 75 seconds

The demo fixture site is the ground truth. It runs on port 4100 with intentional defects for every scanner, a design reference and approved content that differ from it on purpose, and two versions (v1 and v2) so comparisons can be checked on real scans. Tests assert the exact expected results.

Not covered by tests: the live Anthropic and OpenAI APIs and the live Figma API (tested against local mocks only), and the Docker images (not built).

Running and deployment

The same two processes run everywhere; only how they are started changes. You need Node.js 22.12 or newer, PostgreSQL 15 or newer, and Chromium installed through Playwright.

Process	Port	Development	Production
Web app	3000	`npm run dev:web`	`npm run build`, then `npm run start:web`
Worker	3001 (health, localhost only)	`npm run dev:worker`	`npm run start:worker`
Demo fixture site	4100	`npm run demo:site`	Not deployed
PostgreSQL	5432	Local service	Managed or self-hosted database

First-time setup, in order: `npm install` → `npm run setup:env` (creates `.env` with secrets) → `npm run db:bootstrap` (creates the database role and the two databases; asks for the PostgreSQL superuser password) → `npm run db:migrate` → `npx`

`playwright install chromium`. After every code update run `npm run db:migrate` again. `npm run dev` starts the web app and worker together in one terminal.

Production checklist (from `docs/DEPLOYMENT.md`):

- Put the web app behind an HTTPS reverse proxy that overwrites `X-Forwarded-For`, and list it in `TRUSTED_PROXIES`.
- Set `APP_URL` to the public `https://` address; cookies then become Secure and HSTS is sent.
- Close sign-up (`AUTH_ALLOW_SIGNUP=false`) once accounts exist, and empty `SCAN_ALLOWED_PRIVATE_HOSTS`.
- Run one web instance (rate limits are in memory); the worker can scale, since jobs are claimed safely.
- Run the worker in an isolated environment with restricted outbound network access, because it opens untrusted websites in Chromium.
- Back up the database and the `storage/` folder together.

Docker: a `Dockerfile`, `docker-compose.yml` and `deploy.env.example` are in the repository but have never been built or run, because Docker was not available. The same build sequence (`npm ci` → build → prune development packages → start) was verified without Docker.

What is missing or not configured

The core product (crawl, capture, six scanners, comparisons, issues, reports, history) is complete and tested; what is missing falls into five groups, starting with the two integrations that are built but switched off.

1. Built but not configured in this installation

Component	What you see today	What it needs
AI layer	The "AI analysis" step shows Skipped; issues use the scanners' wording and rule-catalogue fix guidance; brand guidelines are read for literal values only	`AI_PROVIDER`, its API key and `AI_MODEL` in `.env`, then a worker restart
Figma API	A Figma link is marked unavailable with guidance; design values come from manual entry or a guideline document	`FIGMA_ACCESS_TOKEN` in `.env`, or a token saved in the project's settings

2. Built but never tested against the real service

- The live Anthropic and OpenAI APIs, including Anthropic's server-side refusal fallback (a beta feature); only local mocks were used.
- The live Figma REST API; only a local mock of its documented responses was used.
- The Docker image and Compose stack; never built or run.
- Shutdown on SIGTERM; it cannot be exercised on Windows and is covered by integration tests only.

3. Planned in Phase 0 but not built (optional items)

- Pixel comparison of Figma frames against screenshots; design checks compare styles only.
- Figma Variables, spacing scales read from Figma, and free-form alignment checks.
- Opt-in checks for exposed files such as `/.git/HEAD` and `/.env`; the security scanner stays passive.
- Scripted login flows for sites behind a login form; only cookies, a custom header or HTTP Basic are supported.
- Cloud file storage (S3); files are on local disk only.
- Organisations and teams; each project belongs to one user.

4. Not planned, but expected in a production product

- Password reset and email verification; the app sends no email.
- Scheduled or recurring scans; every scan is started by hand.
- OCR for scanned PDFs, and tables inside PDFs.
- A comparison section inside reports.
- Automatic removal of orphaned files; `npm run storage:gc` must be run by hand.
- A shared rate-limit store, needed before running more than one web instance.
- A server-side cap on a project's page limit; `SCAN_MAX_PAGES` is only the default (projects allow up to 500).

5. Known open defects (none block normal use)

- Report downloads and artifacts get the app-wide security header instead of their stricter one; the fix belongs in `apps/web/next.config.ts`.
- Brand-guideline extraction skips typography lines written on consecutive lines without a blank line between them.
- In a comparison, a persistent issue that gained a finding on a newly checked page still says "same findings".
- A cancel requested in the last 5 seconds of a scan is not honoured, because the worker checks for cancellation every 5 seconds.

- Rejected requests count toward the per-user rate limits.

Glossary

Term	Meaning in QA Copilot
Project	One website to test, with its settings and references
Scan	One run of the pipeline against a project's website
Step	One stage of a scan (crawl, capture, a scanner, AI, grounding), with its own status and reason
Capture	One page rendered at one screen size: screenshot, HTML, styles, text, measurements
Finding	One problem a scanner saw in one scan, with its evidence
Evidence	The stored proof of a finding: an HTML snippet, a header, a measured value, a screenshot
Verdict	How sure a finding is: confirmed, potential (needs review), inconclusive, unable to verify
Issue	A problem tracked across scans with a stable number (ISSUE-12) and a status people change
Fingerprint	The scanner rule plus what it applies to; it decides that two scans saw the same issue
Reference	What the site should look like (design reference) or say (approved content)
Design reference version	A confirmed, numbered set of design values that scans compare against
Grounding	Checking every AI-written sentence against the stored evidence
Fixture site	The local demo website on port 4100 with known defects, versions v1 and v2
Heartbeat	A timestamp the worker updates every 5 seconds to show a scan is still alive

Term	Meaning in QA Copilot
SSRF	Server-side request forgery: tricking a server into fetching internal addresses; blocked by the target checks
CSP	Content-Security-Policy: a browser rule that only lets the app's own scripts run